

Trabalho Prático – Etapa 1 – GCC130

Analizador Léxico (*Flex*)

Caio Bueno Finocchio Martins

Lana da Silva Miranda

Nina Tobias Novikoff da Cunha Ribeiro

Universidade Federal de Lavras (UFLA)

Professor: Ricardo Terra

1 Introdução

A análise léxica corresponde à primeira fase do processo de compilação. Nessa etapa, o código-fonte é lido e suas sequências de caracteres são agrupadas em unidades léxicas, chamadas *tokens*, que representam elementos básicos da linguagem. Para isso, utilizam-se padrões descritos por expressões regulares, permitindo que o analisador reconheça lexemas válidos e associe a cada ocorrência uma ação específica. Além de preparar a entrada para as próximas fases do compilador, essa etapa também pode registrar identificadores em uma tabela de símbolos e indicar erros léxicos com sua respectiva posição no texto. Essa compreensão foi fundamentada na apostila da disciplina, no material de Niemann sobre *Lex* e *Yacc* e na obra clássica de Aho et al. [2, 3, 1].

Neste projeto, o objetivo da Etapa 1 foi implementar, com o uso do *Flex*, um analisador léxico para uma minilinguagem didática. O *Flex* foi adotado por permitir a descrição dos padrões léxicos por meio de expressões regulares, gerando automaticamente o código correspondente do analisador [2, 3]. O analisador desenvolvido foi responsável por reconhecer declarações de tipos, identificadores, números inteiros e decimais, palavras-chave, operadores relacionais, aritméticos e lógicos, além de símbolos de pontuação. Também fez parte do escopo imprimir o *token* reconhecido, seu lexema e sua posição no arquivo de entrada, desconsiderar espaços em branco e comentários, detectar e reportar erros léxicos ao usuário e exibir a tabela de símbolos construída durante a análise.

2 Decisões de Projeto

As seguintes decisões de projeto foram tomadas para otimizar o desempenho, a funcionalidade e a robustez do analisador léxico. As decisões foram embasadas no conteúdo da disciplina, em materiais de apoio sobre *Lex/Yacc* e em consultas pontuais ao *ChatGPT*, usadas apenas como apoio na organização textual e na revisão conceitual [2, 3, 4]:

- **Inclusão de *Tokens* Adicionais:** Para aumentar a funcionalidade da linguagem e facilitar validações futuras, foram adicionados *tokens* além dos requisitos mínimos.

O operador de atribuição (=) foi incluído para permitir operações de atribuição de valores, e literais, cadeias de caracteres entre aspas, foram definidos para suportar a representação de dados textuais.

- **Estrutura de Dados para a Tabela de Símbolos:** Optamos por implementar uma tabela *hash* com listas encadeadas para gerenciar a tabela de símbolos. Essa escolha se baseou na eficiência da busca e inserção de símbolos. O uso de listas encadeadas foi adotado para tratar colisões, garantindo a integridade e acessibilidade dos dados, mesmo em cenários com alta ocupação da tabela.
- **Tratamento de Palavras Reservadas:** As palavras reservadas da linguagem foram tratadas diretamente por meio de diagramas de transição no analisador léxico, sem serem armazenadas na tabela de símbolos. Essa abordagem evita buscas desnecessárias na tabela para elementos fixos da linguagem, otimizando o reconhecimento léxico e tornando mais fácil distingui-las dos identificadores definidos pelo usuário.
- **Categorização de *Tokens* e Uso de *yylval*:** Os *tokens* foram definidos em categorias gerais, como `OPERADOR_RELACIONAL` e `OPERADOR_ARITMETICO`, e o campo *yylval* foi usado para diferenciar lexemas que pertencem à mesma categoria. Por exemplo, "`<=`" e "`>`" são ambos `OPERADOR_RELACIONAL`, mas seus valores em *yylval* são `OR_LE` e `OR_GT`, respectivamente. No entanto, o operador de atribuição (=) e as palavras reservadas possuem *tokens* individuais para cada um, sendo exceções à regra adotada.
- **Uso de *typedef union* para *yylval*:** Para permitir que *yylval* armazenasse diferentes tipos de dados conforme o lexema analisado, usamos um *typedef union*. Essa estrutura permite que *yylval* adote o tipo de um inteiro, ponto flutuante, caractere ou um ponteiro para a estrutura `Simbolo`, conforme a necessidade do *token*.
- **Definição do *yylval* por Tipo de Lexema:**
 - **Identificadores:** *yylval* armazena um ponteiro para a estrutura `Simbolo` (`*Simbolo`).
 - **Operadores Lógicos, Aritméticos, Relacionais e Tipos de Dados:** *yylval* armazena um valor inteiro (`int`) definido por `#define` que representa cada lexema específico.
 - **Delimitadores:** *yylval* armazena o caractere do delimitador (`char`).
 - **Números Inteiros:** *yylval* armazena o valor inteiro (`int`).
 - **Números Ponto Flutuante:** *yylval* armazena o valor de ponto flutuante (`float`).
 - **Literais:** Nesta etapa, *yylval* não foi definido para literais. Decidiu-se adiar o tratamento do valor semântico de literais para a análise sintática, devido ao problema de vazamento de memória.
- **Tratamento de Erros Específicos com Expressões Regulares:** Foram usadas expressões regulares para identificar e relatar erros léxicos específicos, como comentários de múltiplas linhas não fechados, literais que não terminam em aspas, números com zero à esquerda (012, -06), números de ponto flutuante com vírgula

em vez de ponto (1,23) e identificadores que não começam com o caractere \$. Essa abordagem permite mensagens de erro mais claras para os desenvolvedores.

3 Dificuldades Encontradas

Durante o desenvolvimento do analisador léxico, surgiram desafios que afetaram as decisões e a implementação do projeto. As principais dificuldades incluem:

- **Distinção entre Decisão de Projeto e Expectativas da Etapa:** Foi necessário distinguir se algumas abordagens eram escolhas deliberadas de projeto ou se representavam um desvio das expectativas da fase de análise léxica.
- **Compreensão da Função da Tabela de Símbolos na Análise Léxica:** A função da tabela de símbolos na fase léxica gerou questionamentos, pois, embora seja essencial para o compilador como um todo, sua interação e responsabilidades nessa etapa precisaram ser pesquisadas a fundo para evitar erros por falta de entendimento.
- **Diferenciação no Tratamento de Erros — Léxico vs. Sintático:** Foi importante estabelecer uma clara fronteira entre os erros tratados pelo analisador léxico e aqueles que eram responsabilidade do analisador sintático, de forma a não ignorar erros léxicos nem incluir erros sintáticos nessa etapa.
- **Definição do `yylval` para Diferentes Lexemas:** Atribuir o valor semântico (`yylval`) adequado para cada lexema, considerando a variedade de tipos de *tokens*, foi um desafio técnico. Era necessário um mecanismo flexível para armazenar diferentes tipos de dados, como inteiros, `float`, ponteiros e caracteres, o que exigiu uma solução robusta e eficiente.

4 Diagramas de Transição (*DFA*s)

4.1 *DFA* 1 – Literais

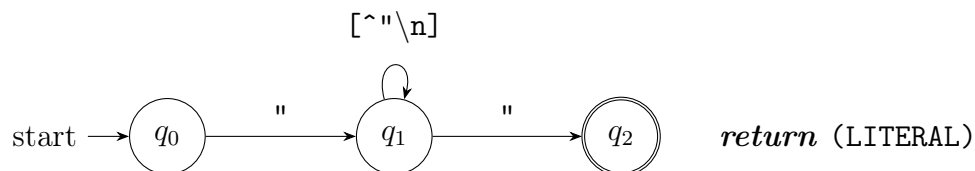


Figura 1: *DFA* para reconhecimento de literais.

4.2 DFA 2 – Identificadores

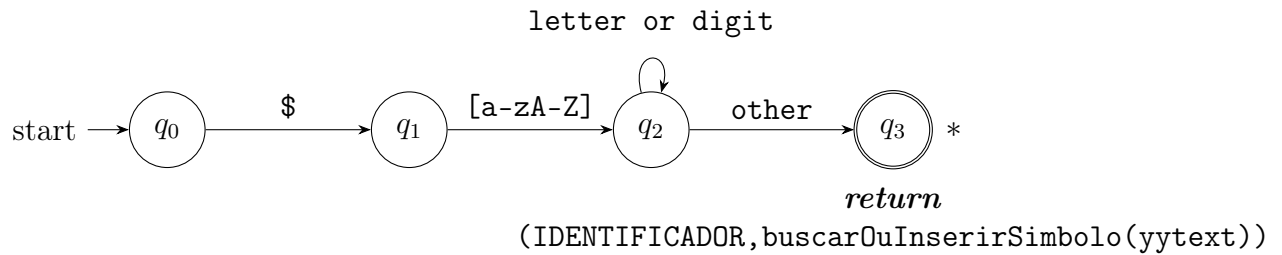


Figura 2: DFA para reconhecimento de identificadores.

5 Arquivo de Teste e Saída Esperada

5.1 Arquivo de Entrada (teste_lexer.txt)

```
1 /* Comentário de múltiplas linhas
2 fechado corretamente */
3 // Comentário de uma linha
4
5
6 int $x = 10;
7 float $y = 25.5;
8
9 $x = $x + 1;
10 $y = $y / 2.0;
11
12 if ($x < 20) {
13     print("texto literal fechado");
14 }
15 else {
16     read($x);
17 }
18
19 while ($x != 0 || $y >= 1.0) {
20     $x = $x - 1;
21     $y = $y * 3.5;
22     return;
23 }
24
25 ( ) { } ; ,
26 < > == <= >= !=
27 + - * /
28 && || !
29 =
30
31 123
32 45.67
33 -99
34 0
35
36 $x
37 $abc123
```

```

38 $A1
39
40 -0
41 001
42 -012
43 -3.14
44 12,34
45 abc
46 "literal aberto
47 @
48
49 /* comentário de múltiplas linhas não fechado

```

Listing 1: Arquivo de teste utilizado

5.2 Saída do Analisador (*stdout*)

LIN	COL	LEXEMA	TOKEN
6	1	int	TIPO_DADO
6	5	\$x	IDENTIFICADOR
6	8	=	OPERADOR_ATRIBUICAO
6	10	10	NUMERO INTEIRO
6	12	;	DELIMITADOR
7	1	float	TIPO_DADO
7	7	\$y	IDENTIFICADOR
7	10	=	OPERADOR_ATRIBUICAO
7	12	25.5	NUMERO FLOAT
7	16	;	DELIMITADOR
9	1	\$x	IDENTIFICADOR
9	4	=	OPERADOR_ATRIBUICAO
9	6	\$x	IDENTIFICADOR
9	9	+	OPERADOR_ARITMETICO
9	11	1	NUMERO INTEIRO
9	12	;	DELIMITADOR
10	1	\$y	IDENTIFICADOR
10	4	=	OPERADOR_ATRIBUICAO
10	6	\$y	IDENTIFICADOR
10	9	/	OPERADOR_ARITMETICO
10	11	2.0	NUMERO FLOAT
10	14	;	DELIMITADOR
12	1	if	PALAVRA_RESERVADA
12	4	(	DELIMITADOR
12	5	\$x	IDENTIFICADOR
12	8	<	OPERADOR_RELACIONAL
12	10	20	NUMERO INTEIRO
12	12	)	DELIMITADOR
12	14	{	DELIMITADOR
13	5	print	PALAVRA_RESERVADA
13	10	(	DELIMITADOR
13	11	"texto literal fechado"	LITERAL
13	34	)	DELIMITADOR
13	35	;	DELIMITADOR
14	1	}	DELIMITADOR
15	1	else	PALAVRA_RESERVADA
15	6	{	DELIMITADOR
16	5	read	PALAVRA_RESERVADA

41	16	9	(	DELIMITADOR
42	16	10	\$x	IDENTIFICADOR
43	16	12	)	DELIMITADOR
44	16	13	;	DELIMITADOR
45	17	1	}	DELIMITADOR
46	19	1	while	PALAVRA_RESERVADA
47	19	7	(	DELIMITADOR
48	19	8	\$x	IDENTIFICADOR
49	19	11	!=	OPERADOR_RELACIONAL
50	19	14	0	NUMERO INTEIRO
51	19	16		OPERADOR_LOGICO
52	19	19	\$y	IDENTIFICADOR
53	19	22	>=	OPERADOR_RELACIONAL
54	19	25	1.0	NUMERO FLOAT
55	19	28	)	DELIMITADOR
56	19	30	{	DELIMITADOR
57	20	5	\$x	IDENTIFICADOR
58	20	8	=	OPERADOR_ATRIBUICAO
59	20	10	\$x	IDENTIFICADOR
60	20	13	-	OPERADOR_ARITMETICO
61	20	15	1	NUMERO INTEIRO
62	20	16	;	DELIMITADOR
63	21	5	\$y	IDENTIFICADOR
64	21	8	=	OPERADOR_ATRIBUICAO
65	21	10	\$y	IDENTIFICADOR
66	21	13	*	OPERADOR_ARITMETICO
67	21	15	3.5	NUMERO FLOAT
68	21	18	;	DELIMITADOR
69	22	5	return	PALAVRA_RESERVADA
70	22	11	;	DELIMITADOR
71	23	1	}	DELIMITADOR
72	25	1	(	DELIMITADOR
73	25	3	)	DELIMITADOR
74	25	5	{	DELIMITADOR
75	25	7	}	DELIMITADOR
76	25	9	;	DELIMITADOR
77	25	11	,	DELIMITADOR
78	26	1	<	OPERADOR_RELACIONAL
79	26	3	>	OPERADOR_RELACIONAL
80	26	5	==	OPERADOR_RELACIONAL
81	26	8	<=	OPERADOR_RELACIONAL
82	26	11	>=	OPERADOR_RELACIONAL
83	26	14	!=	OPERADOR_RELACIONAL
84	27	1	+	OPERADOR_ARITMETICO
85	27	3	-	OPERADOR_ARITMETICO
86	27	5	*	OPERADOR_ARITMETICO
87	27	7	/	OPERADOR_ARITMETICO
88	28	1	&&	OPERADOR_LOGICO
89	28	4		OPERADOR_LOGICO
90	28	7	!	OPERADOR_LOGICO
91	29	1	=	OPERADOR_ATRIBUICAO
92	31	1	123	NUMERO INTEIRO
93	32	1	45.67	NUMERO FLOAT
94	33	1	-99	NUMERO INTEIRO
95	34	1	0	NUMERO INTEIRO
96	36	1	\$x	IDENTIFICADOR
97	37	1	\$abc123	IDENTIFICADOR
98	38	1	\$A1	IDENTIFICADOR

```

99 40 1 -0 Erro léxico: o valor zero não pode ser negativo
100 41 1 001 Erro léxico: 0 a esquerda
101 42 1 -012 Erro léxico: 0 a esquerda
102 43 1 -3.14 Erro léxico: pontos flutuantes não podem assumir
valores negativos
103 44 1 12,34 Erro léxico: pontos flutuantes devem usar "." e não ","
104 45 1 abc Erro léxico: identificadores devem começar com "$"
105 46 1 "literal aberto Erro léxico: literais devem ser fechados com aspas
106 47 1 @ Erro léxico: caractere inválido
107 50 1 /* comentário de múltiplas linhas não fechado
108 Erro léxico: comentários de múltiplas linhas devem ser fechados com "*/"
109
110
111 Tabela de Símbolos
112 HASH LEXEMA TOKEN QUANT. OCORRENCIAS
113 -----
114 24 $x 18 9
115 25 $y 18 6
116 40 $abc123 18 1
117 98 $A1 18 1
118 -----

```

Listing 2: Saída gerada pelo analisador léxico

6 Conclusão

Este trabalho permitiu transformar conceitos teóricos da análise léxica em uma implementação concreta, tornando mais claro como o compilador começa, de fato, a interpretar um programa. Ao longo da etapa, foi possível observar que escolhas aparentemente simples influenciam diretamente a clareza da solução e a forma como o sistema poderá evoluir depois.

Mais do que chegar a um resultado funcional, esta etapa serviu para consolidar a lógica por trás do reconhecimento de padrões, da identificação de erros e da organização das informações obtidas durante a leitura do código-fonte. Com isso, foi possível concluir esta etapa de forma satisfatória, unindo prática e teoria no desenvolvimento do analisador léxico.

7 Referências

- [1] AHO, Alfred V.; LAM, Monica S.; SETHI, Ravi; ULLMAN, Jeffrey D. *Compilers: Principles, Techniques, and Tools*. 2. ed. Boston: Pearson/Addison-Wesley, 2006.
- [2] TERRA, Ricardo. *Compiladores*. Apostila/slides da disciplina GCC130, Universidade Federal de Lavras (UFLA), 2026.
- [3] NIEMANN, Tom. *Lex & Yacc Tutorial*. Epaperpress, s.d. Disponível em: <https://epaperpress.com/lexandyacc/>.
- [4] OPENAI. *ChatGPT*. Ferramenta de inteligência artificial utilizada como apoio para revisão textual, organização do relatório e esclarecimentos conceituais. Acesso em: 2026.